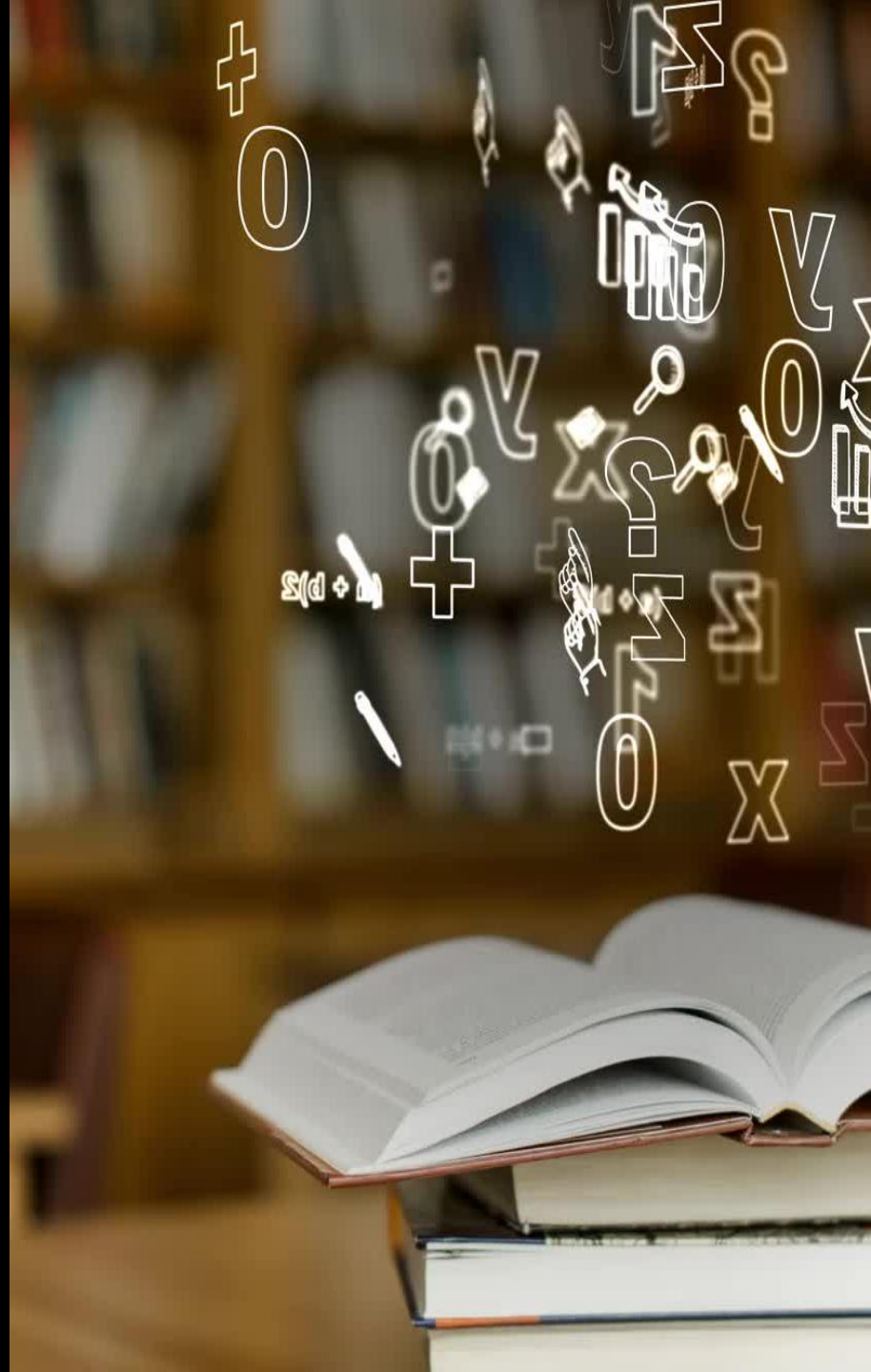


DEFAULT DICTIONARIES

AUTOMATIC ERROR HANDLING

Default Dictionary

- A default dictionary is a specialized subclass of Python's built-in dict class.
- Default dictionaries comes from the collections module, and it provides all the standard dictionary functionality with an extra feature.
- Default dictionaries allows you to define a default value that will be returned if you try to access or modify a key that isn't in the dictionary.



Standard Dictionary

This example produces a KeyError for coordinate ‘z’

```
try:
    coordinates = { 'x':100, 'y':200 }
    print( coordinates['x'], coordinates['y'], coordinates['z'] )
except KeyError as e:
    print(f'KeyError: {e}')
```

Prints KeyError: ‘z’

Default Dictionary

The default dictionary prints a ‘0’ for coordinate ‘z’

```
from collections import defaultdict
coordinates = defaultdict(int, {'x':100, 'y':200})
print(coordinates['x'], coordinates['y'], coordinates['z'])
```

Prints: 100 200 0

Flexible Default Values

- You can provide any callable as the default factory, so default values can be anything from simple integers to more complex objects.

```
coordinates = defaultdict(int, { 'x':100, 'y':200 })
```

The **INT** in this case defaults to ZERO. A zero will be substituted when an **KeyError** would normally be produced.

Summary

- Overall, default dictionaries make handling missing keys and initializing values much more efficient and less error-prone.

- They're especially useful in scenarios where you need to aggregate or group data.

